

React.js

Day 2 — Hooks, useState, Props & Conditional Rendering

Student Notes · Beginner Friendly · Interview Ready

Rendering & Re-rendering | State | Hooks | useState | Props | Conditional Rendering

What You Will Learn

- What is Rendering and Re-rendering
- Why Local Variables don't update the UI
- What is State
- Hooks in React
- useState Hook
- Props
- Conditional Rendering

1. Rendering & Re-rendering

How React displays and updates the UI

Term	Meaning	Example
Rendering	Displaying the UI on the screen for the first time	Page first loads
Re-rendering	Updating the UI when data changes	Counter value increases, form input updates, theme changes

Note: React automatically re-renders a component whenever its state changes.

2. Local Variables vs State

Why normal variables can't update the UI

The Problem with Local Variables

- A local variable is a regular JavaScript variable
- React does NOT track local variables
- Even if the value changes, the UI does not update

```
let count = 0;

count++; // value changes internally...
        // but React has no idea – UI stays the same
```

The Solution — State

- State is data that React actively tracks
- Whenever state changes, React automatically updates the UI
- State makes the application reactive and interactive

Common Examples of State

State Example	What it Controls
Counter value	Number displayed on screen
Form input	Text typed in an input box
Toggle button	ON/OFF switch
Dropdown menu	Open/closed dropdown
Dark/Light theme	Background and text color

3. Stateful vs Stateless Components

Two types of components based on data management

Stateful Component

- Manages its own data using state
- Re-renders when state updates
- Examples: Counter, Login Form, Toggle Button, Shopping Cart

Stateless Component

- Does NOT manage any data
- Only displays information passed via props
- Examples: Header, Footer, Static Text, Logo

Note: Earlier, only Class Components could use state. Now, Functional Components can also use state using Hooks (introduced in React 16.8).

4. Hooks

Special functions that give Functional Components superpowers

Hooks are special functions provided by React that allow Functional Components to use React features like state and lifecycle — previously only available in Class Components.

Introduced in

- React version 16.8

Commonly Used Hooks

Hook	Purpose
useState()	Store and update state
useEffect()	Handle side effects (API calls, timers)
useRef()	Access DOM elements directly
useMemo()	Cache expensive calculations
useCallback()	Cache functions to avoid re-creation
useContext()	Share data across components without props

Custom Hooks

- You can also create your own hooks
- Custom hooks start with the word 'use' by convention — e.g. useFetch(), useForm()

5. useState Hook

Managing state in Functional Components

useState is a Hook that lets you add and manage state inside a Functional Component. It returns the current state value and a function to update it.

Syntax

```
const [state, setState] = useState(initialValue);
```

Breakdown

Part	Meaning	Example
state	Current value of state	count
setState	Function used to update the value	setCount
initialValue	Starting value when component mounts	0

Example — Counter

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Updating State — Correct vs Wrong

✓ **Correct — always use the setter function**

```
setCount(count + 1);
```

✗ **Wrong — never modify state directly**

```
count = count + 1; // React won't detect this change
```

Why useState is Important

- React tracks the value automatically
- UI updates instantly when state changes
- Makes the application interactive

What Happens When setState() is Called?

```
      setState() is called
            ↓
    React updates the state value
            ↓
      Component re-renders
            ↓
    Updated UI appears on screen
```

6. Props

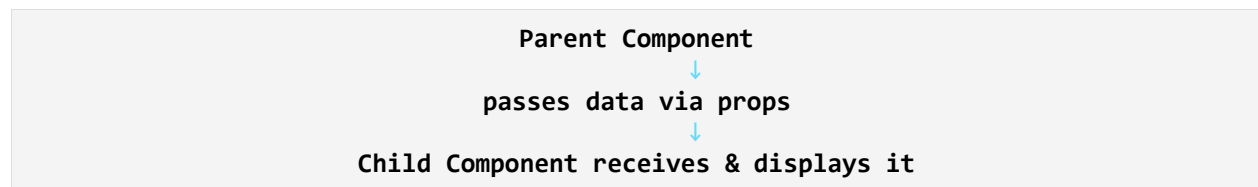
Passing data from Parent to Child components

Props (short for Properties) are used to pass data from one component to another. They help create reusable and dynamic components.

Key Facts About Props

- Props are read-only — child components cannot modify them
- Props always flow in one direction: Parent → Child
- Props are passed as attributes on JSX elements
- Inside the component, props arrive as an object

One-Way Data Flow



Example — Passing Props

Parent sends data

```
<Counter diff={5} />
<Counter diff={10} />
<Counter diff={1} />
```

Child receives data

```
function Counter({ diff }) {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + diff)}>
      +{diff}
    </button>
  );
}
```

Props are Objects Internally

React stores all props as a single object.

```
// React receives this internally:
props = {
  diff: 5,
  name: 'Yathe'
}
```

Multiple Props

Parent

```
<Card
  title="React"
  author="John" />
```

Child — destructuring props directly

```
function Card({ title, author }) {
  return <h2>{title} by {author}</h2>;
}
```

Props vs State

State	Props
Managed inside the component itself	Passed from the Parent component
Can be updated with setter function	Read-only — cannot be modified by child
Used for dynamic, changing data	Used to configure and customise components
Triggers re-render on change	Re-renders only when parent updates them

7. Conditional Rendering

Showing or hiding UI based on a condition

Conditional Rendering means displaying different UI elements based on a condition — showing what is needed and hiding what is not.

Real-World Examples

Condition	UI Behaviour
isLoggedIn = true	Show Logout button
isLoggedIn = false	Show Login button
showPassword = true	Show the password text
isLoading = true	Show spinner / skeleton
isLoading = false	Show actual content
isDark = true	Apply dark background

Three Ways to Write Conditional Rendering

Method 1 — if...else

```
if(isLogin) {  
  return <button>Logout</button>;  
}  
else {  
  return <button>Login</button>;  
}
```

- Clearest and most readable
- Best when the logic is complex or spans multiple lines

Method 2 — Ternary Operator (Most Common in JSX)

```
isLogin ? <button>Logout</button> : <button>Login</button>
```

- Most popular — works inline inside JSX
- Short form of if...else
- Best for simple two-way conditions

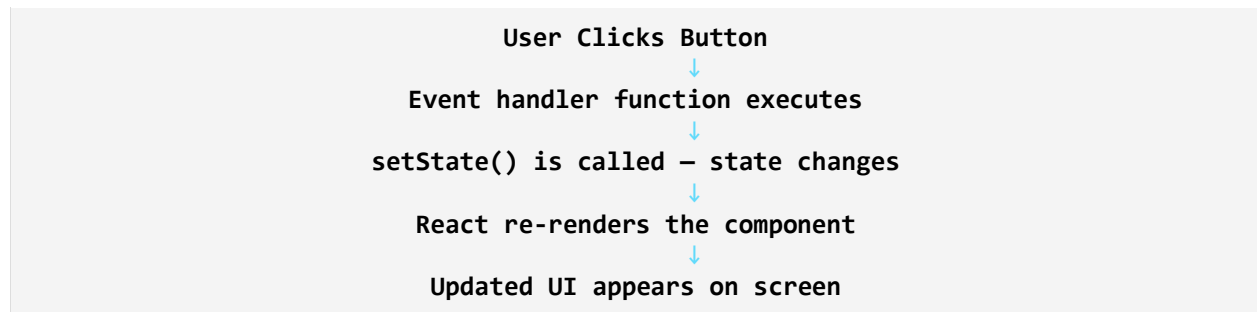
Method 3 — Logical AND (&&)

```
showCounter && <Counter />  
  
// If showCounter is true → renders <Counter />  
// If showCounter is false → renders nothing
```

- Best when you want to show something OR show nothing
- There is no 'else' — use ternary when you need a fallback

Method	Use When	Has Else?
if...else	Complex multi-line logic	Yes
Ternary (?:)	Simple two-way condition in JSX	Yes
Logical &&	Show or show nothing	No

Toggle Workflow — How Most React Interactions Work



This exact pattern is behind most interactive React features — counters, toggles, theme switchers, login/logout, and more.

Common Beginner Mistakes

- ✗ Using a normal variable instead of state — UI won't update
- ✗ Updating state directly: `count++` instead of `setCount(count + 1)`
- ✗ Forgetting to import `useState` at the top of the file
- ✗ Trying to modify props inside a child component
- ✗ Assuming props can send data from Child to Parent — they cannot
- ✗ Writing very complex conditions directly inside JSX — move to a variable

Real-World Uses

Concept	Real-World Use Cases
useState	Counter, Like button, Form handling, Shopping cart, Theme switcher
Props	Product cards, User profiles, Reusable buttons, Blog cards, Dashboard widgets
Conditional Rendering	Authentication UI, Loading screens, Popups, Notifications, Dropdown menus

Mini Practice Tasks

Task 1 — Full Counter

- Increment button: count + 1
- Decrement button: count - 1
- Reset button: count = 0

Task 2 — Counter Using Props

- Create the Counter component once
- Use the same component with diff={1}, diff={5}, diff={10}

```
<Counter diff={1} />  
<Counter diff={5} />  
<Counter diff={10} />
```

Task 3 — Show / Hide Counter

- A button that shows or hides the Counter
- Use Logical AND (&&) for conditional rendering

Task 4 — Light / Dark Theme Toggle

- Click to switch between light and dark background
- Use useState to track the current theme
- Apply class based on state value

Task 5 — Login / Logout Button

- One button that changes its label
- isLogin true → show Logout button
- isLogin false → show Login button
- Use the Ternary Operator inside JSX

Interview Questions & Answers

Q: What is Rendering in React?

A: Rendering means displaying the UI on the screen.

Q: What is Re-rendering?

A: Re-rendering means updating the UI when state or props change.

Q: Why doesn't React update the UI when a local variable changes?

A: Because React does not track local variables. Only state changes trigger re-renders.

Q: What is State?

A: State is data managed by React that automatically re-renders the UI when it changes.

Q: What is a Hook?

A: A Hook is a special React function that lets Functional Components use React features like state and lifecycle.

Q: What is the purpose of useState?

A: useState is used to create and update state inside a Functional Component. It returns the current value and a setter function.

Q: What happens when setState() is called?

A: React updates the state value and re-renders the component so the UI reflects the change.

Q: What are Props?

A: Props are used to pass data from a Parent Component to a Child Component. They are read-only.

Q: Can a Child Component modify Props?

A: No. Props are read-only (immutable). Only the Parent can control the props it sends.

Q: What is Conditional Rendering?

A: Displaying different UI elements based on a condition — show what is needed, hide what is not.

Q: Name three ways to perform Conditional Rendering.

A: 1. if...else 2. Ternary Operator (? :) 3. Logical AND (&&)

Q: Difference between State and Props?

A: State is managed inside the component and can be updated. Props are passed from parent and are read-only. Both can trigger re-renders.

Key Takeaways

- ✓ Rendering displays the UI — re-rendering updates it when data changes.
- ✓ React does NOT track local variables — changing them will not update the UI.
- ✓ State is used to store dynamic data that React tracks and reacts to.
- ✓ useState Hook — returns [currentValue, setterFunction] with an initial value.
- ✓ Always use the setter function to update state — never modify state directly.
- ✓ Hooks allow Functional Components to use state and lifecycle (since React 16.8).
- ✓ Props pass data from Parent → Child — always one-way, always read-only.
- ✓ Props are objects internally — destructure them for cleaner code.
- ✓ Conditional Rendering: if...else for complex logic, ternary for two-way, && for show-or-nothing.
- ✓ React apps become interactive by combining State + Props + Conditional Rendering.